EE-170L Computer Programming Lab

Laboratory Session 11

Strings in C++

Name:	Shahzad Gul
Semester:	2
Section:	B
Reg No:	0671
Lab:	11
Date:	Spring 2026

1. Objective

The objective of this lab session is to understand and apply the concepts of strings in C++ programming. Students will learn about C-strings (C-style strings) and string objects from the Standard C++ Library string class. By the end of this lab, students will be able to declare, initialize, manipulate, and perform operations on strings using built-in functions.

Key learning outcomes include:

- Understanding C-string declaration and initialization
- Working with string objects in C++
- Using built-in string functions such as length() and concatenation
- Reading strings with spaces using getline() and cin.get()

2. Theory

2.1 C-Strings (C-Style Strings)

In C programming, a collection of characters is stored in the form of arrays. This concept is also supported in C++ programming, which is why it is called C-strings. C-strings are arrays of type **char** terminated with a null character **\0** (ASCII value 0). The C++ compiler automatically places the null character at the end of the string when it initializes the array.

Examples of declaring and initializing C-strings:

```
char myString[] = "Hello";
char myString[6] = "Hello";
char myString[] = {'H','e','l','l','o','\0'};
char myString[100] = "Hello";  // Extra space allocated
```

2.2 String Object

In C++, you can also create a string object for holding strings. Unlike char arrays, string objects have no fixed length and can be extended as needed. The **string** class is part of the Standard C++ Library and provides many useful member functions for string manipulation.

To read text containing blank spaces, the **getline()** function is used instead of **cin >>**:

```
string str;
getline(cin, str);  // Reads entire line including spaces
```

2.3 Common String Functions

C++ supports a wide range of functions that manipulate null-terminated strings. The following table summarizes the most commonly used string functions:

Function	Example	Description
strcpy()	strcpy(s1, s2);	Copies string s2 into string s1
strcat()	strcat(s1, s2);	Concatenates string s2 onto the end of string s1
strlen()	strlen(s);	Returns the length of string s
strcmp()	strcmp(s1, s2);	Returns 0 if s1 and s2 are the same; less than 0 if s1 < s2; greater than 0 if s1 > s2
strrev()	strrev(s);	Reverses the given string
strlwr()	strlwr(s);	Converts string to lowercase
strupr()	strupr(s);	Converts string to uppercase

3. Lab Tasks

Task 1: String Length Calculation

Objective: Write a C++ program that declares and initializes a string variable called **message** with a phrase of your choice. Use a built-in function to calculate and display the length of the string.

Source Code:

```
/*
 * Name: Shahzad Gul
 * Semester: 2
 * Section: B
 * Reg No: 0671
 * Lab: 11 - Task 1: String Length Calculation
 * Description: Program that declares and initializes a string variable
 *              and uses a built-in function to calculate and display
 *              the length of the string.
 */

#include <iostream>
#include <string>  // Required for string class
using namespace std;

int main()
{
    // Declare and initialize a string variable with a phrase
    string message = "Welcome to UET Peshawar";

    // Display the original string
    cout << "Original string: " << message << endl;

    // Use built-in function to calculate and display the length of the string
    cout << "Length of the string: " << message.length() << endl;

    return 0;
}
```

Expected Output:

```
Original string: Welcome to UET Peshawar
Length of the string: 23
```

Explanation: The program declares a string variable **message** and initializes it with "Welcome to UET Peshawar". It then uses the built-in **length()** member function of the string class to calculate the number of characters in the string, including spaces. The result is displayed along with the original string.

Task 2: String Concatenation

Objective: Declare two string variables called **firstName** and **lastName** and initialize them with your first and last name. Concatenate the two strings to form a full name and display the result.

Source Code:

```
/*
 * Name: Shahzad Gul
 * Semester: 2
 * Section: B
 * Reg No: 0671
 * Lab: 11 - Task 2: String Concatenation
 * Description: Program that declares two string variables for first and
 *              last name, concatenates them to form a full name,
 *              and displays the result.
 */

#include <iostream>
#include <string>  // Required for string class
using namespace std;

int main()
{
    // Declare and initialize two string variables
    string firstName = "Shahzad";
    string lastName = "Gul";

    // Declare a string to store the concatenated full name
    string fullName;

    // Display the original strings
    cout << "First Name: " << firstName << endl;
    cout << "Last Name: " << lastName << endl;

    // Concatenate the two strings to form a full name
    fullName = firstName + " " + lastName;

    // Display the concatenated full name
    cout << "Full Name: " << fullName << endl;

    return 0;
}
```

Expected Output:

```
First Name: Shahzad
Last Name: Gul
Full Name: Shahzad Gul
```

Explanation: The program declares two string variables **firstName** and **lastName**, initialized with "Shahzad" and "Gul" respectively. It then concatenates them using the **+** operator

with a space in between to form the full name. The **+** operator for strings allows easy concatenation without needing functions like strcat().

4. Conclusion

In this lab session, we successfully explored the concept of strings in C++. We learned the difference between C-strings (char arrays) and string objects from the C++ Standard Library. The lab tasks demonstrated practical applications of string manipulation, including:

- Using the **length()** function to determine the number of characters in a string
- Using the **+** operator to concatenate two strings with a separator
- Understanding the advantages of string objects over C-strings (dynamic sizing, built-in functions)

String objects provide a more convenient and safer way to handle text data in C++ compared to C-strings, as they automatically manage memory and provide a rich set of member functions for common operations. These skills are fundamental for developing applications that require text processing, user input handling, and data formatting.

5. References

- Deitel, P. & Deitel, H. (2017). *C++ How to Program* (10th ed.). Pearson.
- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
- UET Peshawar, EE-170L Computer Programming Lab Manual, Spring 2026.